Q → frozen sets ?

Q → Default dict ?

Q → json strings ?

Functions

default argument

```
def greet (name = 'Krish'):
    print ('Hello', name)

greet ()
```

⇒ Hello Krish.

---

another eg.

default values

```
def sums (a=1, b=2):
    return a + b
sums (3)
```

⇒ 5 → since a=3 & b is missing.

Positional args

positional args

```
def a (*b):
    return b
```

OR `def a (*args):`

⇒ returns all the arguments that have been passed into a.

Keyword Args

```
def a(** k_args):
    for key,value in k_args.items():
        print(key,value)

a(name='Krish', age=3)
```

=>
```
name : Krish
age  : 3
```

* We can use positional & keyword arguments together.

eg. `def a(*args, ** k_args):`

but order of input of positional ~~arg~~ args should always be before k_args.

---

return statements → used to return a value from a function.

Q → in k_args, why do we not use keys in double quotes?

① SQL, ADF → sunkalp

In-built functions

→ isdigit() → checks if the string contains any digits

→ isupper()
→ islower()
→ ischar() — ?

→ any() → returns True if any of the condition in the bracket is True.

→ isalphanumeric()

eg: if any (char in '&#@^%' for char in password)

→ .copy() → to create a shallow copy of an object.

→ ~~str.replace(~~

→ str.replace('a', 'b') → replaces all 'a's in the given string to 'b's

---

→ str.split() → splits the string based on the argument.

→ ~~str.join~~

→ str.strip() → strips the spaces from the ends (or any argument passed)

→ 'x'.join() → joins 2 arguments with 'x' in middle.

→ dict.get(word, 0) → returns value of word, else 0

Reading a file

```
with open(file_path, 'r') as file:
    for line in file:
        words = line.split().strip()

    return ...
file.close()
```

More fun()

→ sum()

→ max()

→ min()

→ upper()

→ lower()

→ floor()

etc